TRABAJO BASE DE DATOS (1er CUATRIMESTRE)

GESTIÓN DE DATOS PARA UNA CADENA DE TIENDAS DE COSMÉTICOS



"Potencia tu Tienda de Cosméticos con Inteligencia de Datos"

ALUMNO: ALICIA CASAUX VÁZQUEZ

PROFESOR: LUIS VALENCIA CABRERA

GRADO: ESTADISTCA

CURSO: 2024-2025

UNIVERSIDAD D SEVILLA 1505

CONTENIDO

INTRODUCCIÓN 3

DISEÑO CONCEPTUAL 3

DISEÑO CONCEPTUAL EN DIA 5

DISEÑO LÓGICO 6

DISEÑO LÓGICO EN MYSQL WORKBENCH 10
CONSULTAS 11

ACCESS 16

FORMULARIOS 16

MONGODB 19

CONSULTAS 20
OPERACIONES DE CONSULTA BÁSICAS *20*
OPERACIONES DE MANIPULACIÓN DE DATOS *22*
OPERACIONES EMPLEANDO EL PIPELINE DE AGREGACIONES *23*

INTRODUCCIÓN

En la era digital actual, la gestión eficiente de la información es crucial para el éxito de cualquier negocio, y las tiendas de cosméticos no son una excepción. En el sector de la cosmética, donde la oferta es amplia y variada, contar con un sistema de base de datos bien estructurado puede marcar la diferencia en la organización de inventarios, seguimiento de ventas y la relación con los clientes.

DISEÑO CONCEPTUAL

Se quiere desarrollar una base de datos sobre una cadena de tiendas de cosméticos, teniendo en cuenta los siguientes requisitos, realiza el diseño conceptual.

- Se deberán almacenar las distintas tiendas de cosméticos de esta cadena, identificadas por el nombre de la ciudad dónde se sitúan, y de los que se almacenará su dirección, email y teléfono.

- La tienda vende diferentes tipos de productos cosméticos con su número de producto (único), nombre del producto, descripción, marca, precio y tipo de piel/cabello. Los productos pertenecen a categorías (perfume, maquillaje, cuidado facial, cuidado corporal) debemos conocer su código (único) y nombre. Un producto pertenece a una categoría y cada categoría tiene muchos productos. A su vez cada categoría puede pertenecer a otra categoría/subcategoría (por ejemplo: las categorías, pintalabios, colorete pertenecen a la categoría maquillaje).

- En la tienda trabajan empleados identificados por su número de empleado (único), nombre, apellidos y edad. Los empleados pueden ser vendedores, maquilladores y encargados de inventario. Además, algunos empleados pueden supervisar a otros empleados, un empleado solo puede supervisar a uno a la vez, mientras que un empleado puede ser supervisado por varios.

- Se ha determinado que se debe almacenar la información de los proveedores que distribuyen los productos, conocemos el código del proveedor(único), el nombre de la empresa y el teléfono de contacto. Los encargados de inventario son aquellos que piden a los proveedores la cantidad de productos que necesitan para la tienda, en una fecha determinada.

- El vendedor se encarga de vender los productos de la tienda que son comprados por clientes. Además, los clientes pueden puntuar los productos (una nota del 0 al 5) y poner comentarios. Únicamente podemos tener una nota asignada por cliente al producto. También, interesaría conocer la nota media de cada producto.

- Los clientes son identificados por su DNI, nombre, apellidos, ciudad, teléfono y tipo de piel/cabello. Algunos clientes puede ser vip y premium. Los clientes vip tienen promociones especiales y los clientes premium tienen descuentos.

- Los pagos que realizan los clientes hay que almacenarlos, para ello conocemos el número de transacción (único), la forma de pago (efectivo, tarjeta, …) y la fecha en la que se realizó el pago.

- Los clientes pueden pedir que sean maquillados por los maquilladores de la tienda, su servicio tiene una duración, precio y fecha. Los maquilladores deberán tener hecho un curso para poder maquillar a los clientes.

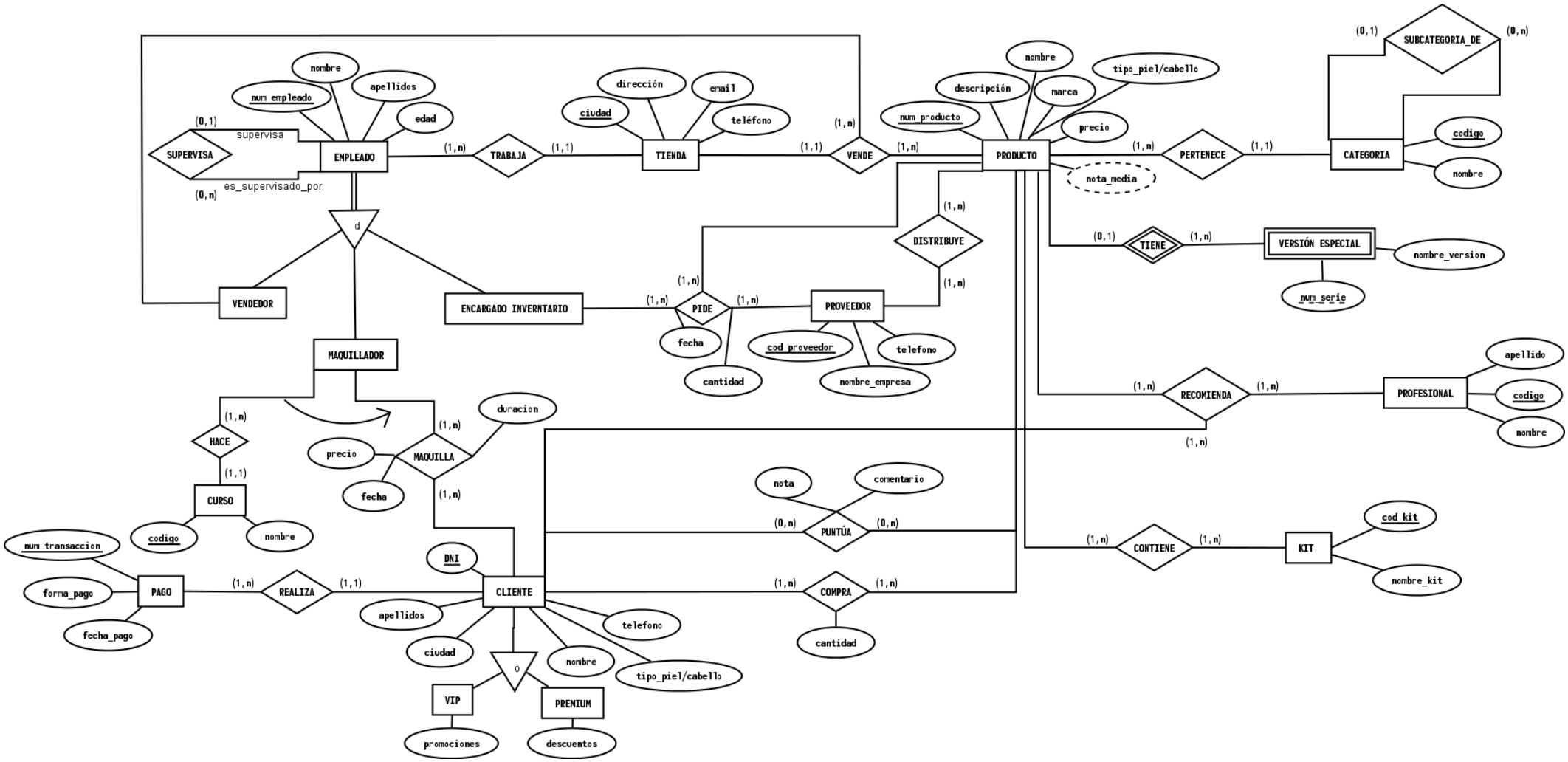
- Algunos productos son versiones especiales de otros productos. Por ejemplo, un perfume es edición limitada del perfume base y se distinguen por un número de serie y tienen un nombre especial.

- Los kits de maquillaje están identificados por un código y nombre, estos kits contienen una serie de productos.

- Los profesionales identificados por un código, nombre y apellido recomiendan a los clientes una serie de productos según su tipo de piel o cabello.

Se ha diseñado este diseño conceptual con la herramienta DIA. El archivo que lo contiene es DC tienda.dia, en la carpeta DIA.

DISEÑO CONCEPTUAL EN DIA



Para los empleados de la tienda hemos hecho una jerarquía total y exclusiva ya que el trabajador solo puede pertenecer obligatoriamente a un subtipo.

Para los clientes se utiliza una jerarquía parcial y solapada, ya que un cliente no es obligatoriamente de un subtipo y además si pertenece al subtipo puede ser de varios.

Las relaciones de maquillador son inclusivas, ya que: un maquillador solo puede maquillar a un cliente si ha hecho al menos un curso. NOTA: *Esto solo se especifica en el diseño conceptual para el resto del trabajo no se tomará en cuenta.*

En los productos, se tiene para cada producto la nota media de cada uno, esta nota es un atributo derivado.

Versión especial es una entidad débil en identificación es necesario saber el código del producto al que hace referencia.

DISEÑO LÓGICO

TIENDA (ciudad, dirección, email, teléfono)
- PK: ciudad

CATEGORIA (cod_categoria, nombre, cod_cat_sub)
- PK: cod_categoria
- FK: cod_cat_sub —> CATEGORIA (cod_categoria)

PRODUCTO (num_producto, descripción, nombre, marca, precio, tipo_piel_cabello, cod_categoria)
- PK: num_producto
- FK: cod_categoria —> CATEGORIA (cod_categoria)

VERSION_ESPECIAL (num_serie,nombre_especial,num_producto)
- PK: num_serie, num_producto
- FK: num_producto —> PRODUCTO (num_producto)

EMPLEADO (num_empleado, nombre, apellidos, edad, ciudad, num_empleado_supervisa)
- PK: num_empleado
- FK: num_empleado_supervisa —> EMPLEADO (num_empleado)

VENDEDOR (num_empleado)
- PK: num_empleado
- FK: num_empleado —> EMPLEADO (num_empleado)

CURSO (código, nombre)
- PK: código

MAQUILLADOR (num_empleado, codigo_curso)
- PK: num_empleado, codigo_curso
- FK: num_empleado —> EMPLEADO (num_empleado)
- FK: codigo_curso —> CURSO (codigo)

ENCARGADO_INVENTARIO (num_empleado)
- PK: num_empleado
- FK: num_empleado —> EMPLEADO (num_empleado)

PROVEEDOR (cod_proveedor, nombre_empresa, teléfono)
- PK: cod_proveedor

PROFESIONAL (codigo,nombre,apellido)
- PK: código

CLIENTE (DNI, nombre, apellidos, teléfono, ciudad, tipo_piel_cabello)
- PK: DNI

VIP (DNI_cliente, promociones)
- PK: DNI_cliente
- FK: DNI_cliente —> CLIENTE (DNI)

PREMIUM (DNI_cliente, descuentos)
- PK: DNI_cliente
- FK: DNI_cliente —> CLIENTE (DNI)

KIT (cod_kit,nombre_kit)
- PK: cod_kit

VENDE (num_empleado, num_producto, ciudad_tienda)
- PK: num_empleado, num_producto, ciudad
- FK: num_empleado —> EMPLEADO (num_empleado)
- FK: num_producto —> PRODUCTO (num_producto)
- FK: ciudad_tienda —> TIENDA (ciudad)

DISTRIBUYE (cod_proveedor, num_producto)
- PK: cod_proveedor, num_producto
- FK: cod_proveedor —> PROVEEDOR (cod_proveedor)
- FK: num_producto —> PRODUCTO (num_producto)

RECOMIENDA (codigo_profesional, num_producto, DNI_cliente)
- PK: codigo_profesional, num_producto, DNI_cliente
- FK: codigo_profesional —> PROFESIONAL (codigo)
- FK: num_producto —> PRODUCTO (num_producto)
- FK: DNI_cliente —> CLIENTE (DNI)

CONTIENE (num_producto, cod_kit)
- PK: num_producto, cod_kit
- FK: num_producto —> PRODUCTO (num_producto)
- FK: cod_kit —> KIT (cod_kit)

PUNTUA (num_producto, nota, comentario, DNI_cliente)
- PK: num_producto, DNI_cliente
- FK: num_producto —> PRODUCTO (num_producto)
- FK: DNI_cliente —> CLIENTE (DNI)

COMPRA (DNI_cliente, num_producto, cantidad)
- PK: num_producto, DNI_cliente
- FK: num_producto —> PRODUCTO (num_producto)
- FK: DNI_cliente —> CLIENTE (DNI)

PAGO (num_transaccion, forma_pago, fecha_pago, DNI_cliente)
- PK: num_transaccion, DNI_cliente
- FK: DNI_cliente —> CLIENTE (DNI)

MAQUILLA (num_empleado, DNI_cliente, fecha, precio, duracion)
- PK: num_empleado, DNI_cliente
- FK: num_empleado —> EMPLEADO (num_empleado)
- FK: DNI_cliente —> CLIENTE (DNI)

PIDE (num_empleado, num_producto, cod_proveedor, cantidad, fecha)
- PK: num_empleado, num_producto, cod_proveedor
- FK: num_empleado —> EMPLEADO (num_empleado)
- FK: num_producto —> PRODUCTO (num_producto)
- FK: cod_proveedor —> PROVEEDOR (cod_proveedor)

Para las jerarquías, se descarta la opción Orientada a Objetos al estar todo el peso de las interrelaciones en la entidad padre.

En el caso de EMPLEADO la única opción más lógica es la directa del DER.

Mientras que, en lugar de las entidades CLIENTE, PREMIUM y VIP como están indicadas arriba usando la opción directa del DER, podríamos recurrir a la opción de tabla única estableciendo como única relación la siguiente:

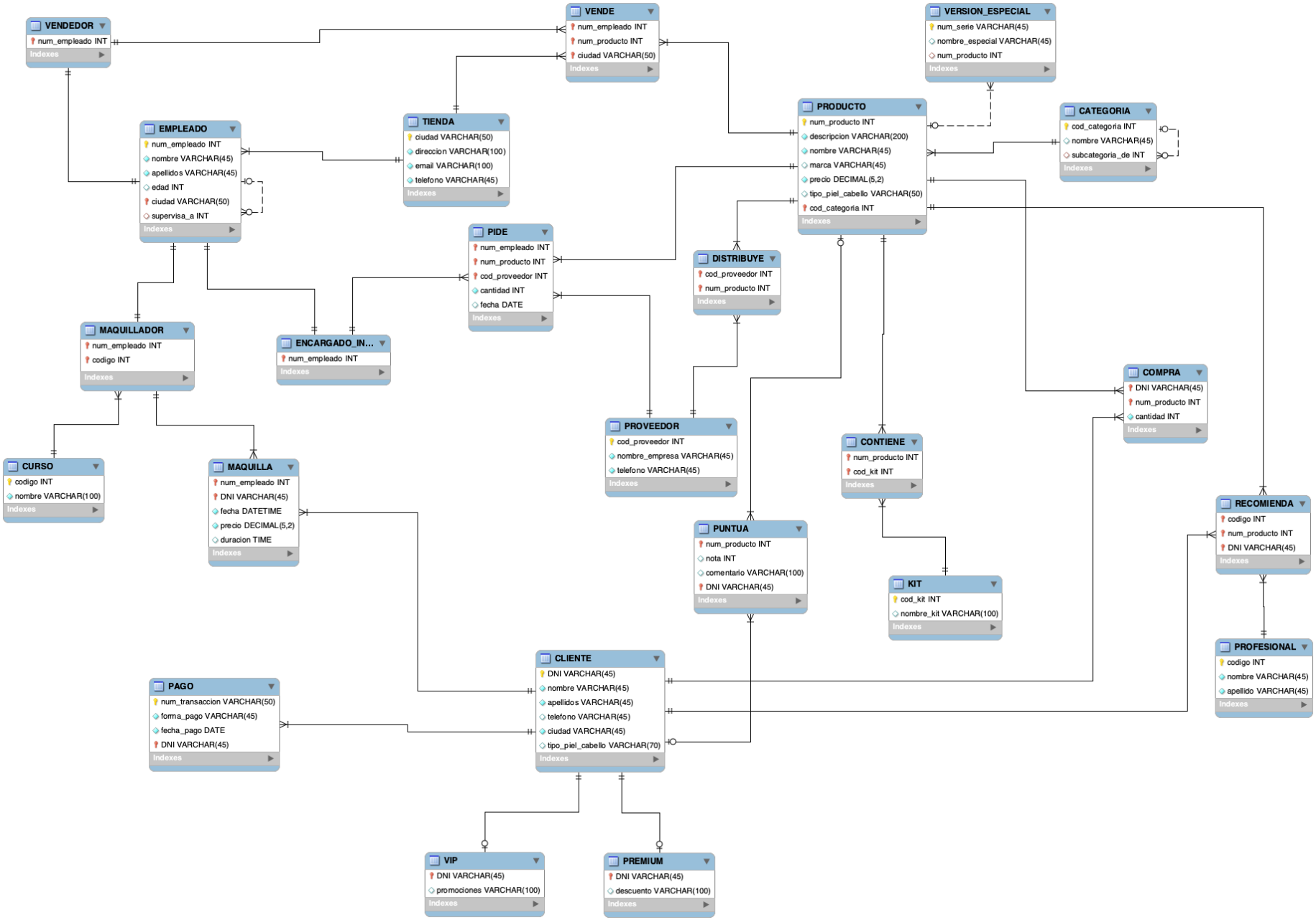
CLIENTE (DNI, nombre, apellidos, teléfono, ciudad, tipo_piel_cabello, promociones, descuentos)
- PK: DNI

cod_cat_sub hace referencia a la propia relación por la interrelación reflexiva subcategoria_de

num_empleado_supervisa hace referencia a la propia relación por la interrelación reflexiva supervisa

Se ha usado el programa MySQLWorkbench para la creación de la bbdd, el diseño lógico anteriormente planteado y las consultas de datos. Todos los archivos se encuentran en una carpeta denominada MYSQLWorkbench

DISEÑO LÓGICO EN MYSQL WORKBENCH



CONSULTAS

1. Mostrar la ciudad y dirección de todas las tiendas registradas.

```
select ciudad, direccion

from tienda;
```

2. Mostrar el número de empleado, nombre y apellidos de los empleados que supervisan a otros.

```
select num_empleado, nombre, apellidos

from empleado

where supervisa_a is not null;
```

3. Obtener el nombre y ciudad de los clientes VIP.

```
select nombre, ciudad

from cliente natural join vip;
```

4. Obtener el código y el nombre de todos los cursos disponibles para maquilladores.

```
select*from curso;
```

5. Mostrar el codigo de la categoria y su nombre, ordenadas alfabeticamente por nombre. Mostrar las 5 primeras.

```
select cod_categoria, nombre

from categoria

order by nombre asc

limit 5;
```

6. Mostrar el nombre, marca y precio de los productos cuyo precio sea mayor a 50.

```
select nombre, marca, precio

from producto

where precio > 50;
```

7. Mostrar los empleados que trabajan en la ciudad de Barcelona.

```
select*from empleado where ciudad = "Barcelona";
```

8. Mostrar el número, nombre y precio de los productos que pertenecen a la categoría con código 15.

```
select num_producto, nombre, precio

from PRODUCTO

where cod_categoria = 15;
```

9. Mostrar el nombre de los cursos cuyo nombre no terminen por "a".

```
select nombre

from curso

where nombre not like "%a";
```

10. Obtener el nombre y descripción de los productos recomendados por el profesional con código 1011.

```
select nombre, descripcion

from producto inner join recomienda using (num_producto)

where codigo = 1011;
```

11. Obtener el nombre y apellidos de los clientes que han puntuado productos con una calificación mayor a 4, junto con el nombre del producto y su comentario.

```
select cliente.nombre, apellidos, producto.nombre as producto, comentario
from cliente inner join puntua on cliente.dni=puntua.dni
                    inner join producto on puntua.num_producto = producto.num_producto
where nota > 4;
```

12. Mostrar el nombre, apellidos y el total de productos adquiridos por cada cliente.

```
select nombre, apellidos, sum(cantidad) as total_productos
from cliente natural join compra
group by dni, nombre, apellidos;
```

13. Mostrar el número de empleado de los maquilladores que están inscritos en más de un curso.

```
select num_empleado
from maquillador
group by num_empleado
having count(codigo) > 1;
```

14. Mostrar el nombre de los productos incluidos en el kit con código 300.

```
select nombre
from producto natural join contiene
where cod_kit = 25;
```

15. Obtener los nombres y descripciones de los productos que no han sido vendidos en ninguna tienda.

```
select nombre, descripcion

from producto

where not exists (select*from vende where producto.num_producto = vende.num_producto);
```

16. Obtener los clientes que no son VIP ni PREMIUM. Mostrar su DNI, nombre y apellidos.

```
select dni, nombre, apellidos

from cliente

where dni not in(select dni from vip)

and dni not in (select dni from premium);
```

17. Obtener los productos recomendados por profesionales que no han recomendado productos a clientes con estatus VIP.

```
select*from producto

where num_producto in (select num_producto

                       from recomienda

                       where recomienda.dni not in (select dni from vip));
```

18. Diseña una tabla llamada DEVOLUCION para registrar las devoluciones de productos realizadas por los clientes. La tabla debe contener las siguientes columnas:

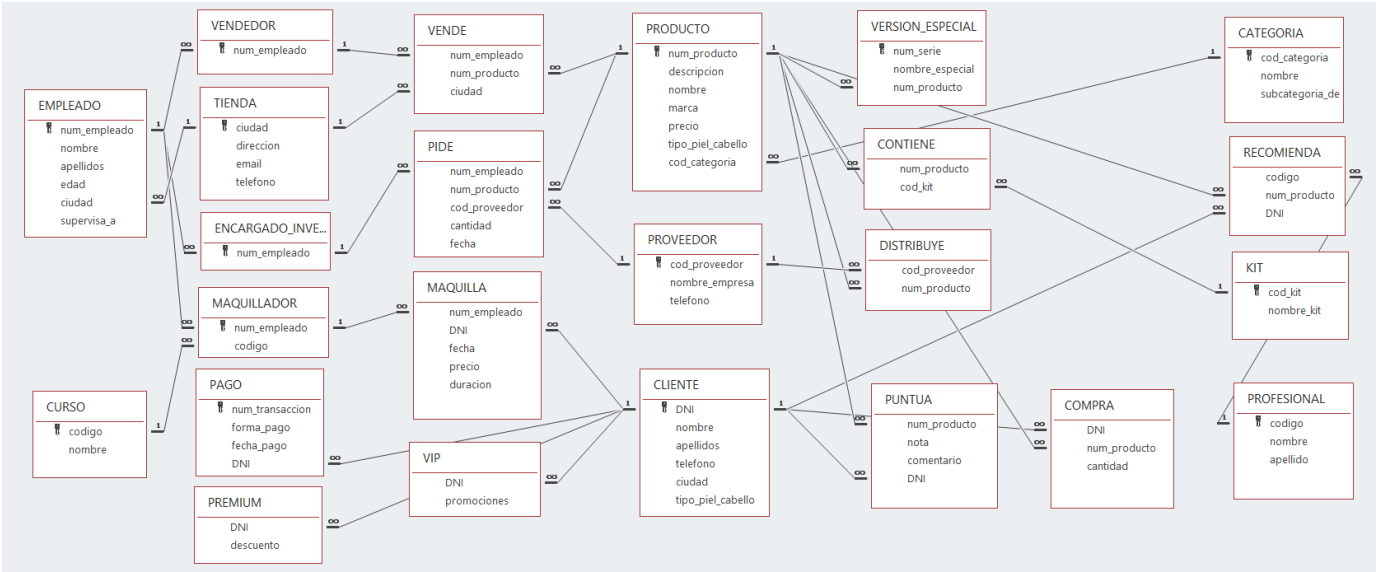
- num_transaccion (INT): El número de transacción asociado a la devolución.
- num_producto (INT): El número del producto que está siendo devuelto.
- DNI_cliente (VARCHAR(20)): El DNI del cliente que realiza la devolución.
- cantidad (INT): La cantidad de productos devueltos.
- fecha_devolucion (DATE): La fecha en la que se realiza la devolución.

```
CREATE TABLE DEVOLUCION (
   num_transaccion INT,
   num_producto INT,
   DNI_cliente VARCHAR(20),
   cantidad INT,
   fecha_devolucion DATE,
   PRIMARY KEY (num_transaccion, num_producto, DNI_cliente),
   FOREIGN KEY (num_producto) REFERENCES PRODUCTO(num_producto),
   FOREIGN KEY (DNI_cliente) REFERENCES CLIENTE(DNI)
);
```

19. Actualiza los precios de todos los productos de la categoria 15 en un 10% (aumentar el precio un 10%).

```
update producto
set precio=precio*1.1
where cod_categoria = 16;
```

ACCESS



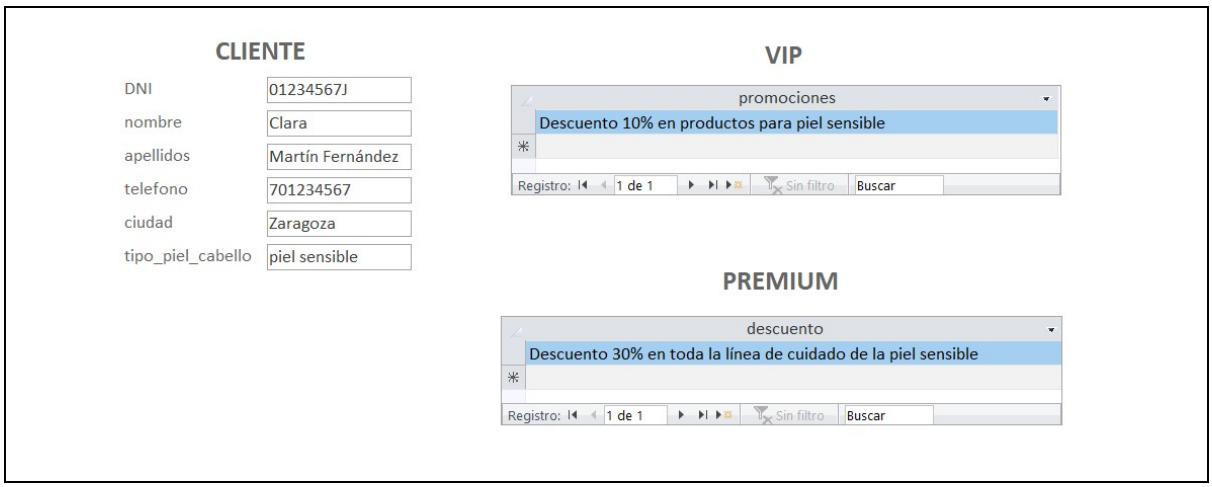
Para implementar en Access la base de datos, hemos creado las mismas tablas con sus respectivas columnas y formatos que en mysql workbench. Para la creación de claves externas se ha usado el asistente de búsqueda, utilizando la integridad de datos.

Desde phpMyAdmin se han exportado en formato csv los datos, y después para poder importarlos en Access se ha tenido que convertir el fromato csv a xlsx.

Los archivos usados se encuentran en la carpeta ACCESS

FORMULARIOS

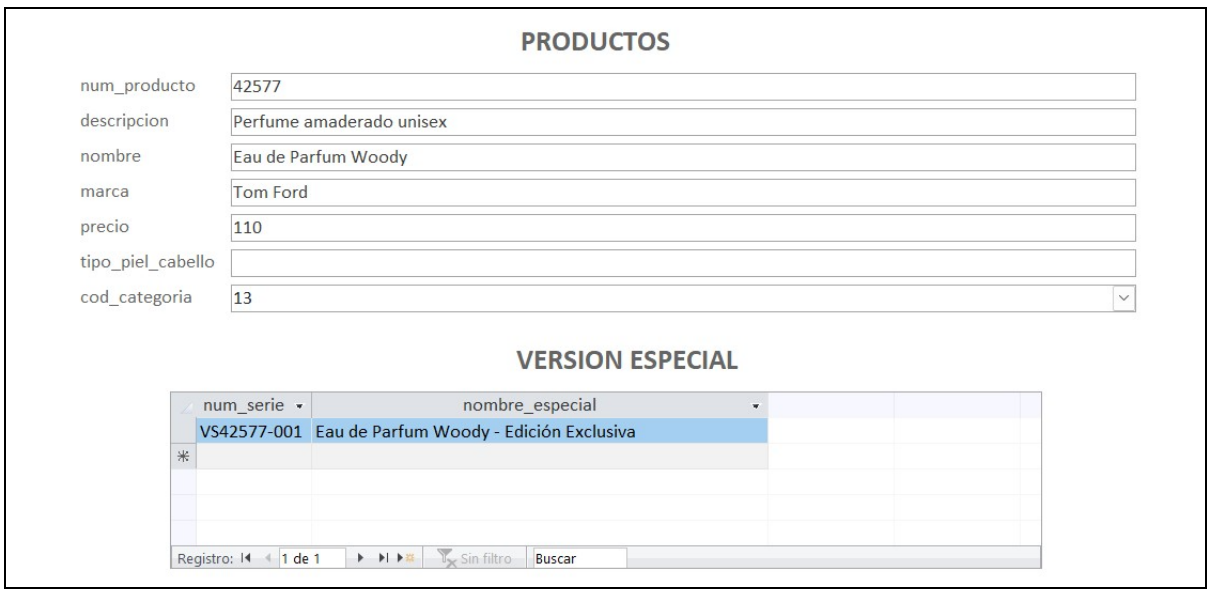
- **TIPO DE CLIENTE**: es un formulario que permite asignar o modificar las promociones y descuentos dependiendo si el cliente es vip, premium o ninguno de estos.



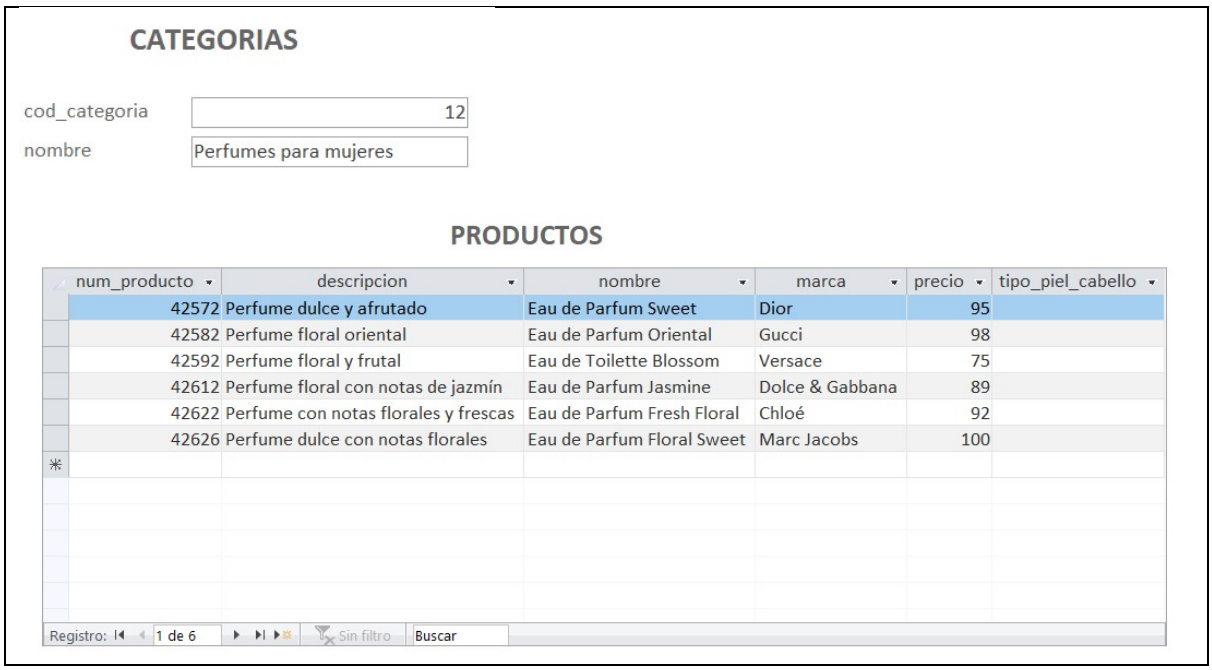
- **EMPLEADOS**: es un formulario que nos permite ver la información de las personas que trabajan en cada tienda.



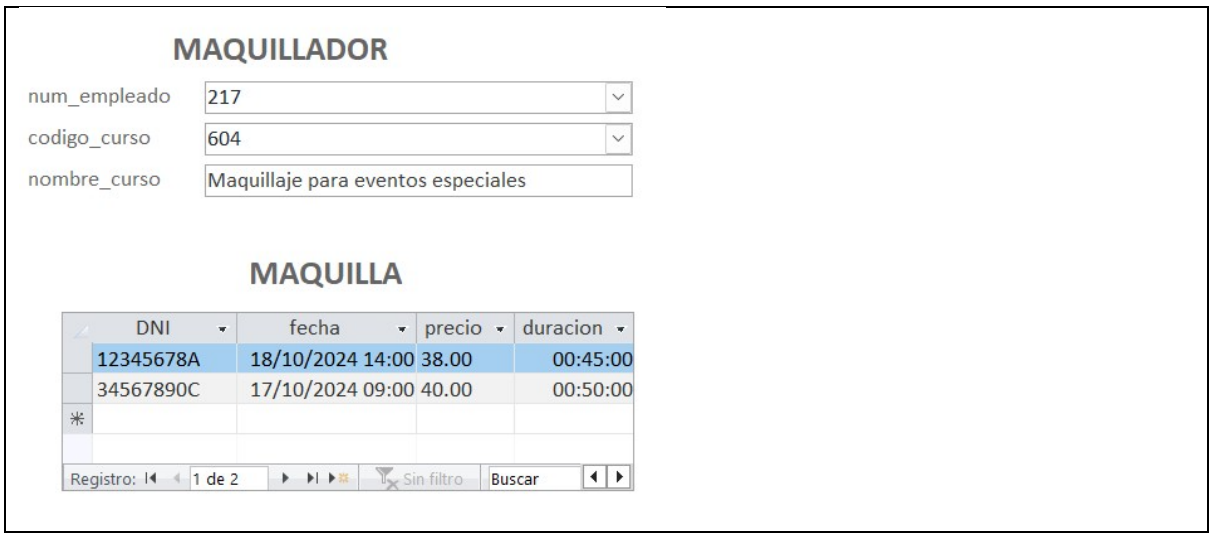
- **PRODUCTO**: en este formulario vemos la información de cada producto y si tiene versión especial.



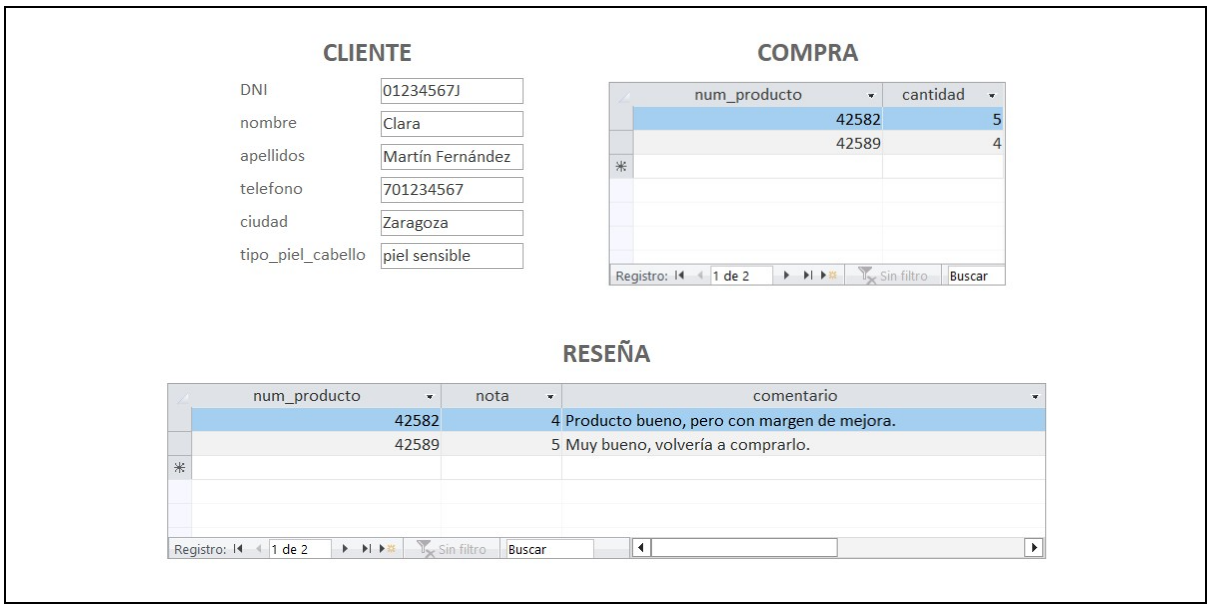
- PRODUCTOS POR CATEGORIA: en este formulario se agrupan los productos según a la categoría que pertenecen.



- CITA MAQUILLADOR: es un formulario que permite ver o asignar cuando un maquillador atiende a un cliente. En una fecha y hora especifica, cuánto durará el servicio y su precio.



- REGISTROS COMPRAS: en este formulario se registran los productos y cantidad que compra un cliente. Además, se puede registrar la reseña que el cliente le da a dicho producto.



MONGODB

En esta base de datos solo hemos trasladado una parte del problema. Para ello, se han creado dos colecciones:

- CLIENTES: se ha trasladado toda la información de los clientes. También se ha añadido los datos vip y premium. Información incluida:
    - _id
    - DNI
    - Nombre
    - Apellidos
    - Teléfono
    - Ciudad
    - tipo_piel_cabello
    - tipo: que a su vez recoge:
        - vip
        - premium

- PRODUCTOS: se ha trasladado toda la información de los productos. También se ha añadido los datos del kit y de las versiones especiales. Información incluida:
    - _id
    - num_producto
    - nombre
    - marca

- precio
- descripción
- tipo_piel_cabello
- cod_categoria
- kits: que a su vez recoge:
  - cod_kit
  - nombre_kit
- versiones_especiales: que a su vez recoge:
  - num_serie
  - nombre_especial

Desde phpMyAdmin se han exportado en formato json los datos, como los datos eran de diferentes tablas se ha modificado los archivos json principales para que la información sea coherente, es decir:

- Al archivo json de clientes se ha añadido los datos de vip y premium
- Al archivo json de productos se ha añadido los datos del kit y versión especial

Para importarlos en mongo se ha usado:

- mongoimport -d trabajo -c productos --file PRODUCTO.json –jsonArray
- mongoimport -d trabajo -c clientes --file CLIENTE.json --jsonArray

Los archivos usados se encuentran en la carpeta MONGODB

NOTA: *las colecciones clientes y productos no contiene todos los datos usados en SQL por tanto cuando hagamos las consultas en mongo algunos resultados no tendrán sentido.*

CONSULTAS

OPERACIONES DE CONSULTA BÁSICAS

1. Devolver los datos de un cliente cuyo DNI tenga la letra Z.

```
db.clientes.findOne({ "DNI": { $regex: /Z/ } })
```

2. Devuelve todos los valores distintos que existen para el campo tipo_piel_cabello.

```
db.clientes.distinct("tipo_piel_cabello", {"tipo_piel_cabello": {$ne: null}})
```

3. Número de clientes que tienen el cabello fino.

db.clientes.find({"tipo_piel_cabello": "cabello fino"}).count()

db.clientes.countDocuments({"tipo_piel_cabello": "cabello fino"})

4. Devolver el nombre y apellidos de los clientes que contengan en su apellido la S. Ordenar alfabéticamente por apellido.

db.clientes.find({"apellidos":{$regex:/S/}},{"nombre":true,"apellidos":true,"_id":false}).sort({"apellidos": 1})

5. Devolver el nombre de los 3 primeros productos que tengan un precio mayor a 100.

db.productos.find({ "precio": { $gt: 100 } },{"nombre":true, "_id":false}).limit(3)

6. Devolver alfabéticamente el nombre de los productos que tengan version especial. Mostrar los 5 primeros.

db.productos.find({"versiones_especiales":{$exists:true,$ne:null}},{"nombre":true,"versiones_especiales.nombre_especial":true, _id:false}).sort({"nombre":1}).limit(5)

7. Numero de productos que pretenecen a un kit.

db.productos.find({"kits": {$exists:true, $ne: null}}).count()

db.productos.countDocuments({"kits": {$exists:true, $ne: null}})

8. Devolver los productos cuyo precio sea mayor a 100 o su categoría sea "13".

db.productos.find({$or: [{ "precio": { $gt: 100 } },{ "cod_categoria": "13" }]})

OPERACIONES DE MANIPULACIÓN DE DATOS

1. Actualiza el número de teléfono de la persona con el DNI "01234567T" a "900123456".

```
db.clientes.updateOne(
 { "DNI": "01234567T" },
 { $set: { "telefono": 900123456 } })
```

2. Inserta un nuevo documento con los mismos campos que el original, pero con el DNI "98765432A" y el nombre "Carlos".

```
db.clientes.insertOne({
 "DNI": "98765432A",
 "nombre": "Carlos",
 "apellidos": "Pérez Gómez",
 "telefono": 801234568,
 "ciudad": "Barcelona",
 "tipo_piel_cabello": "piel grasa",
 "tipo": {
  "vip": null,
  "premium": null
 }
})
```

3. Actualiza todos los documentos de la ciudad "Girona" para que el valor del campo tipo_piel_cabello sea "piel sensible".

```
db.clientes.updateMany({ "ciudad": "Girona" },{ $set: { "tipo_piel_cabello": "piel sensible" } })
```

4. Elimina todos los productos que no tengan ninguna versión especial (versiones_especiales).

```
db.productos.deleteMany({ "versiones_especiales": { $exists: false } })
```

5. Actualiza el nombre del producto con num_producto "42632" a "Sérum Revitalizante Ultra" y su precio a 120.

```
db.collection.updateOne(
 { "num_producto": "42632" },
 { $set: { "nombre": "Sérum Revitalizante Ultra", "precio": 120 } }
)
```

OPERACIONES EMPLEANDO EL PIPELINE DE AGREGACIONES

1. Obtener los tipos de piel/cabello más comunes entre los clientes, mostrando solo aquellos con más de un cliente, y ordenarlos de mayor a menor según su frecuencia.

```
db.clientes.aggregate([
   {$match: {"tipo_piel_cabello": {$ne:null}}},
   {$group: { _id: "$tipo_piel_cabello", total:{$sum:1}}},
   {$match: {total: {$gt:1}}},
   {$sort:{total:-1}])
```

2. Mostrar el número de clientes por ciudad.

```
db.clientes.aggregate([
   {$group: { _id: "$ciudad", num_clientes: { $sum: 1 }}},
   {$sort:{_id: 1}}])
```

3. Calcular el precio promedio de los productos por categoría y mostrar las 3 categorías con el precio promedio más alto.

```
db.productos.aggregate([
 {$match: { "precio": { $exists: true, $ne: null } } },
 {$group: { _id: "$cod_categoria", promedio: { $avg: "$precio" } } },
 {$sort: {promedio: -1 }},
 {$limit:3}])
```

4. Obtener el producto con el precio más bajo para cada categoría, mostrando el nombre del producto y el precio más bajo.

```
db.productos.aggregate([
 { $group: { _id: "$cod_categoria", producto_bajo_precio: { $min: "$precio" },nombre_producto: { $first: "$nombre" }}},
 { $sort: { "_id": 1 } }])
```

5. Calcular el número de productos que contiene los kits y el precio total esperado.

```
db.productos.aggregate([
 { $unwind: "$kits" },
 { $group: {_id: "$kits.nombre_kit",  num_productos: { $sum: 1 },  precio_total: { $sum: "$precio" }}},
 { $sort: { num_productos: -1 } }])
```